

이 문서는 감정 기반 다이어리 서비스 **JustSaying**과 **GPT** 기반 음식 추천 서비스 **gomgom-ai**의 아키텍처 및 개발 내용을 통합 정리한 문서입니다. 두 프로젝트 모두 **FastAPI, Vue, Spring Boot, Redis** 등을 활용하여 풀스택 구조로 설계되었으며, 실제 배포 환경(**AWS EC2/S3/CloudFront**)에 적용되었습니다.

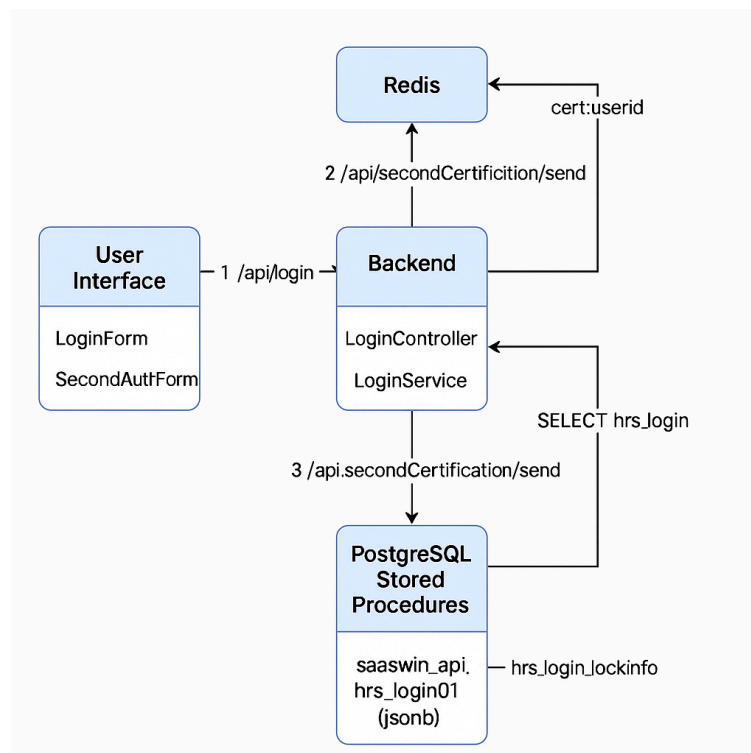
작성자: 민예린 / 개발기간: **2025.04 ~ 진행중**

링크: <https://justsaying.co.kr>

JustSaying 프로젝트 Full Document v1.0

1. API 흐름 개요

JustSaying은 감정 기반 일기 작성, 시간 기록(스탑워치/뱀모도로), 발음 평가 기능, 소셜 로그인 기반 사용자 인증을 제공하는 멀티백엔드 기반 감정 다이어리 서비스입니다.



📌 전체 사용자 흐름 요약

1.1 카카오/네이버 소셜 로그인 흐름

- 사용자가 로그인 버튼 클릭
- 인가 코드 발급 (카카오/네이버)
- **access token** 발급 요청
- 사용자 정보 조회
- 자체 **JWT access/refresh** 토큰 발급 및 쿠키 저장

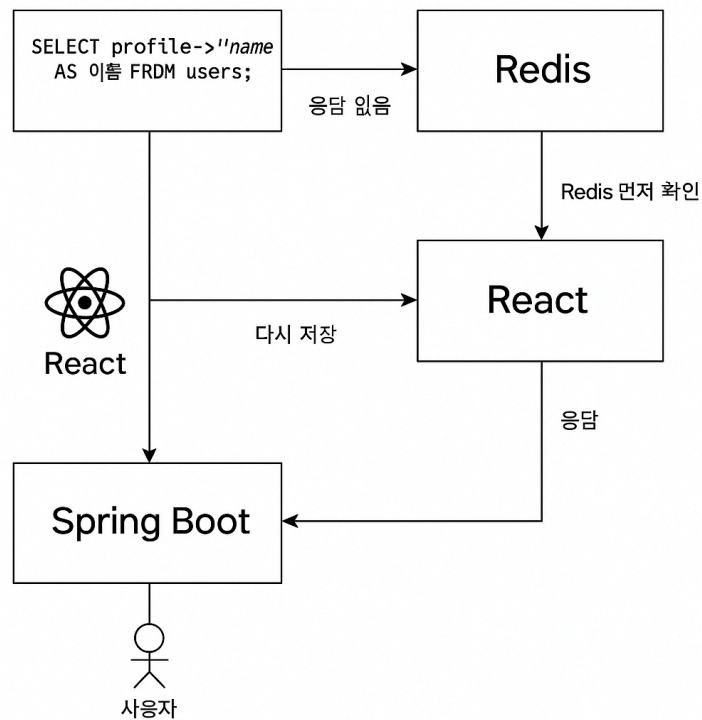
1.2 일기 작성 및 분석 흐름

- 사용자가 감정 일기 작성
- 작성된 일기는 **DB**에 저장
- 동시에 **FastAPI**로 텍스트 전달 → 형태소 분석 + 감정 번역 + 발음 교정
- 번역된 문장과 오디오(**TTS**)는 **Redis**에 캐싱

1.3 시간 기록 (스탑워치/뽀모도로)

- 사용자가 시간 측정 **UI**로 기록
- 완료 시 **session_time**, **duration** 등을 **DB**에 기록
- 카카오톡 나에게 보내기로 기록 공유 가능

2. Redis 캐시 구조 요약



항목	키 구조 예시	TTL	설명
TTS 오디오 캐시	<code>tts_audio:{hash}</code>	1시간	동일 문장의 음성 재사용
번역 결과 캐시	<code>translation:{hash}</code>	1시간	동일 입력 텍스트 재번역 방지
리프레시 토큰	<code>refresh_token:{email}</code>	7일	Redis에 저장된 refreshToken 비교용

캐시는 모두 `hashlib.md5(text.encode()).hexdigest()` 방식으로 키 구성

- **TTS** 캐시: 동일 텍스트 음성 재생산 방지 (1시간 TTL)

- 번역 결과 캐시: 중복 번역 방지 (1시간 TTL)
 - 리프레시 토큰 저장 (선택): 7일 TTL
-

3. 기능별 시퀀스 다이어그램

3.1.1 사용자 인증 및 로그인 시퀀스

sequenceDiagram

participant FE as Frontend

participant BE as Spring Boot

participant OAuth as Kakao/Naver

FE->>OAuth: 소셜 로그인 요청

OAuth-->>FE: 인가 코드 전달

FE->>BE: 인가 코드 전달

BE->>OAuth: access token 발급 요청

OAuth-->>BE: access token 응답

BE->>OAuth: 사용자 정보 조회

OAuth-->>BE: 사용자 정보 반환

BE->>BE: DB에 사용자 저장/조회 + JWT 발급

BE-->>FE: accessToken + refreshToken 전달

3.1.2 소셜 로그인

sequenceDiagram

participant FE as Frontend

participant BE as Spring Boot

participant OAuth as Kakao/Naver

FE->>OAuth: 로그인 요청

OAuth-->>FE: 인가 코드

FE->>BE: 인가 코드 전달

BE->>OAuth: access token 요청

OAuth-->>BE: token + 사용자 정보

BE->>BE: DB에 사용자 저장/조회

BE-->>FE: JWT 발급 + 쿠키 전달

3.2 일기 작성 + 분석 + TTS

sequenceDiagram

FE->>SpringBoot: 일기 제출

SpringBoot->>DB: 일기 저장

FE->>FastAPI: 형태소 분석 요청 (/analyze)

FastAPI->>Okt: 형태소 분석 수행

FastAPI-->>FE: 분석 결과 반환

FE->>FastAPI: /translate 요청

FastAPI->>Google API: 한국어 → 영어 → 일본어

FastAPI->>Redis: 캐시 저장

FastAPI-->>FE: 번역된 문장

FE->>FastAPI: /tts_blob 요청

FastAPI->>GCP: 음성 합성 요청

FastAPI->>Redis: 음성 캐시 저장

FastAPI-->>FE: blob 응답

3.3 스탑워치/뽀모도로 기록

sequenceDiagram

FE->>SpringBoot: 뽀모도로 완료 기록

SpringBoot->>DB: pomodoro 테이블 저장

FE->>SpringBoot: 스탑워치 기록 전송

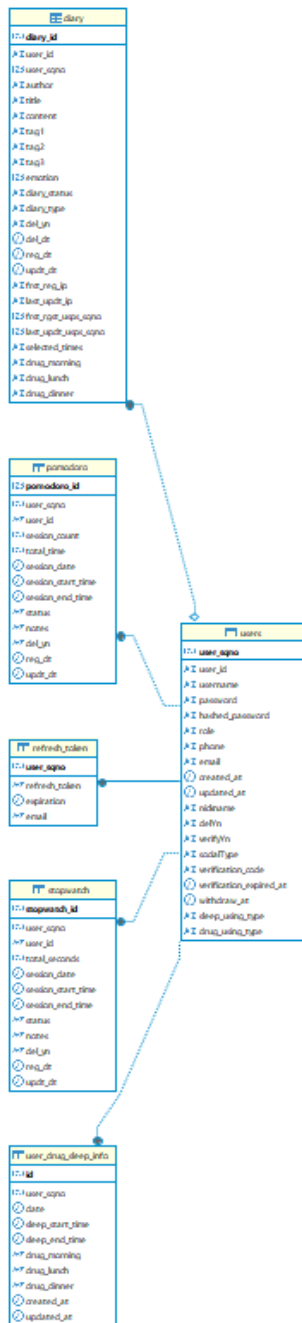
SpringBoot->>DB: stopwatch 테이블 저장

- 위 시퀀스는 **Vue** 프론트엔드 기준이며, **FastAPI**는 주로 **AI** 기능(TTS/번역) 담당, **Spring Boot**는 인증/일기/기록 **API**를 담당합니다.

4. ERD 구조 요약

해당 프로젝트는 사용자의 감정 일기 작성, 시간 관리(스탑워치/뽀모도로), 수면 및 약 복용 정보 추적을 통합 관리하는 플랫폼입니다.

총 **13**개의 테이블로 구성되어 있으며, **JWT** 인증, 이메일 인증, 실시간 채팅, 소셜 로그인 등을 포함합니다.



주요 테이블 목록

- **mt_users**: 사용자 기본 정보 + 소셜타입 + 수면/복약 설정
- **mt_diary**: 일기 내용 + 감정 태그 + 시간/복약 메타 정보 포함

- pomodoro, stopwatch: 시간 기록
- user_drug_sleep_info: 수면/복약 이력 통합
- refresh_token: JWT 토큰 저장
- email_verification: 이메일 인증 코드 저장
- diary_views: 열람 이력
- chats, websocket_connections: 실시간 기능 관련
- user_info: 마케팅 동의, 연령/성별/이름 등 동의 항목

관계 요약

- mt_users 1:N → mt_diary, pomodoro, stopwatch, diary_views
- mt_users 1:1 → refresh_token, user_info
- mt_diary N:M ↔ diary_views

5. 기술 스택 및 구조 설명

영역	기술	설명
프론트엔드	Vue + Pinia + Axios	사용자 UI, API 호출, 쿠키 처리
메인 백엔드	Spring Boot	사용자, 일기, 인증 로직 처리
보조 백엔드	FastAPI	형태소 분석, 번역, TTS 담당

AI 분석	KoNLPy, gTTS, Google TTS, deep_translator	자연어 분석/합성 모듈
캐시/세션	Redis	번역/음성 결과 및 인증 캐시
DB	MySQL	메인 데이터 저장
배포	EC2 + Nginx + CloudFront	서버 및 정적 자원 호스팅

FastAPI는 Vue에서 직접 호출함 (OAuth 외 기능)

6. 추후 완성예정 활용 예시 시나리오

예시 1: 일반로그인과 **SNS** 로그인 미들웨어로 공통처리

- 사용자가 로그인 시도시 3번 틀리면 **Redis**에 캐시하여 사용자 인증 필요 구현
- 같은 사용자일때 일반 로그인이 정지되면 **SNS**로그인 진입 금지 및 알림

예시 2: 감정 일기 작성 + 발음 교정

- 사용자가 감정 상태에 따라 일기 작성
- 작성 후 **FastAPI**로 전달 → 형태소 분석 + 번역 + 발음 평가
- **TTS** 결과로 피드백 오디오 생성
- **Vue**에서 결과 표시 + 오디오 재생

예시 3: **TTS** 재요청 최소화

- 동일 문장은 **Redis**에 캐시
- 동일 문장을 다시 요청해도 **TTS** 재생성 없이 빠른 응답 가능

예시 4: 뽀모도로 분석

- 일일 뽀모도로 총 시간, 횟수 기록
 - 카카오 나에게 보내기로 공유
 - Vue 차트로 시각화
-

7. API 응답 예시 (Swagger 기반)

7.1 일기 작성 성공 시

```
{  
  "success": true,  
  "message": "일기가 저장되었습니다.",  
  "data": {  
    "diaryId": 41,  
    "userSqno": 5,  
    "content": "오늘 너무 피곤했지만 잘 버텼다.",  
    "emotion": "기쁨",  
    "createdAt": "2025-06-18T10:21:00"  
  }  
}
```

7.2 TTS 요청 실패 시

```
{  
  "success": false,  
  "message": "TTS 요청이 비어있거나 오류 발생",  
  "data": null  
}
```

8. 배포스택

JustSaying 배포 아키텍처 설명

JustSaying 프로젝트는 사용자에게 빠르고 안정적인 서비스를 제공하기 위해 AWS 인프라를 활용한 다음과 같은 2단계 배포 구조를 따릅니다.

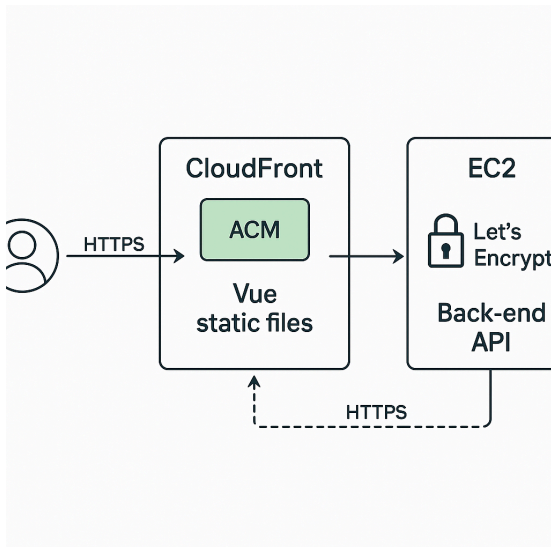
✓ 1단계: 정적 파일 배포 (Vue 프론트엔드)

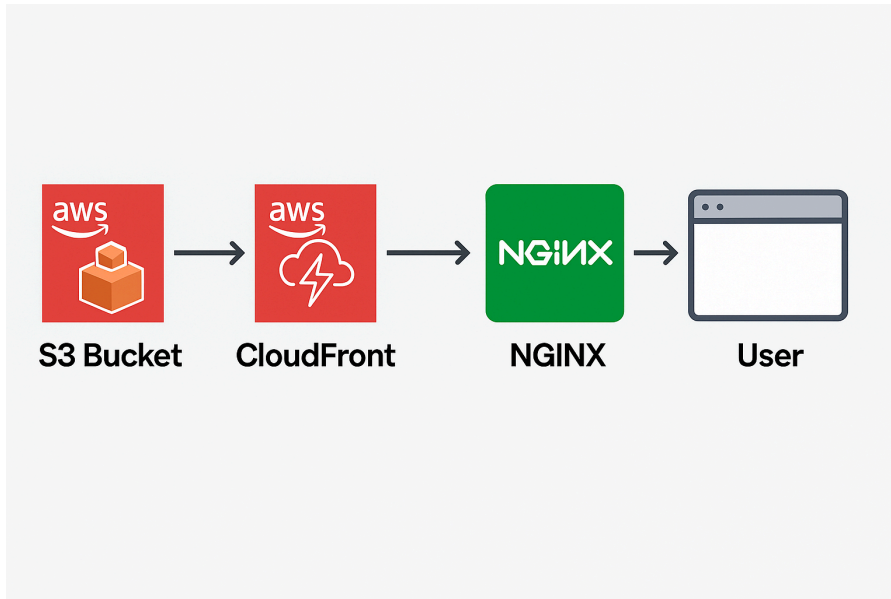
- Vue로 빌드된 정적 파일은 AWS S3에 업로드되어 정적 웹사이트 호스팅 형태로 관리됩니다.

- 이후 CloudFront를 통해 전 세계 사용자에게 빠르게 콘텐츠를 전달하며, 인증서(ACM)를 통해 HTTPS를 지원합니다.

- 사용자의 요청은 CloudFront를 통해 정적 파일을 받아와 브라우저에 렌더링됩니다.

- 정적 파일 안에는 API 호출을 위한 도메인, 카카오 인증 Redirect URI 등이 포함되어 있습니다.





✅ 2단계: 백엔드 API 프록시 처리 (Spring Boot + FastAPI)

- 사용자의 로그인 요청, 일기 저장 요청, 발음 평가 요청 등은 Vue에서 Spring Boot 또는 FastAPI API 서버로 전달됩니다.
- API 요청은 NGINX를 통해 라우팅되며, 도메인 인증은 Let's Encrypt를 통해 이루어집니다.
- Spring Boot 서버는 EC2에 배포되어 JWT 기반 인증, DB 연동, Redis 캐시 처리 등을 담당합니다.
- FastAPI 서버는 발음 분석, 번역, TTS 요청 등을 빠르게 처리하며, 직접 Vue에서 호출됩니다.

💡 요약 흐름

요청 종류

전달 흐름

정적
페이지
요청

S3 → CloudFront → 사용자
브라우저

API 호출
요청
(Spring)

사용자 → NGINX → EC2 (Spring
Boot)

발음 분석
(FastAPI)

사용자 → FastAPI 직접 호출

인증 처리

JWT + 쿠키 기반, RefreshToken은
Redis 활용

작성자: 민예린 / 개발기간: **2025.05** ~ 진행중

링크: <https://mindevprofile.kr>

gomgom-ai 전체 문서 요약 (v1.0)

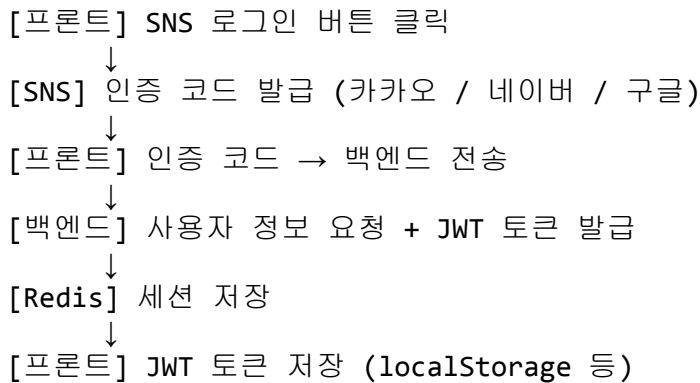
본 문서는 gomgom-ai 프로젝트의 전체 아키텍처, API 흐름, 캐시 구조, 기능별 시퀀스, 데이터베이스 ERD 구조 및 기술 스택 특이사항까지 모두 포함한 정리본입니다.

목차

1. API 흐름 개요
 2. Redis 캐시 구조 요약
 3. 전체 기능별 시퀀스
 4. ERD 구조 요약
 5. 기술 스택 기반 특이사항
 6. 활용 예시
-

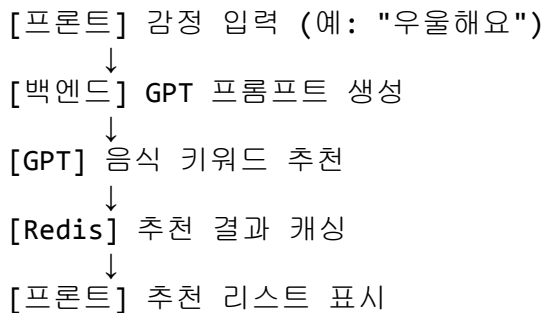
1. API 흐름 개요

✓ 로그인 플로우 (SNS 로그인 연동)



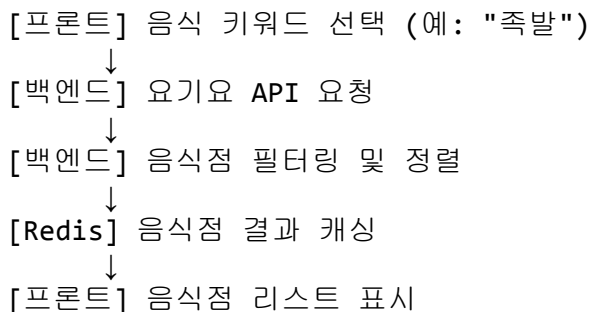
- 사용 API: POST /auth/oauth/callback
 - 사용 기술: OAuth2, JWT, Redis 세션
 - 관련 파일: auth.py, core/oauth_client.py, core/session.py
-

✓ 감정 분석 → GPT 음식 추천



- 사용 API: POST /recommendations/emotion
 - 관련 파일: recommend.py, services/gpt_service.py, core/cache.py
-

✓ 음식점 검색 (요기요 API 활용)



- 사용 API: GET /store/search
 - 관련 파일: services/yogiyo_crawler.py, utils/extract_keywords.py
-

✅ 레시피 및 재료 정보 출력 - 추후 완성 예정

[프론트] 음식 클릭 (예: "김치찌개")

↓

[백엔드] Spoonacular API 호출

↓

[백엔드] 재료 번역 (영어 → 한글)

↓

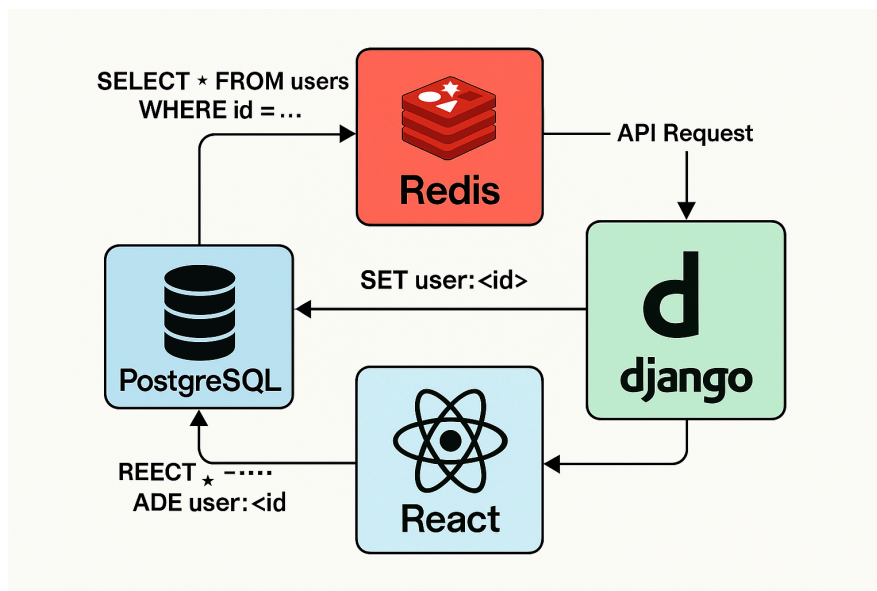
[프론트] 조리법, 재료, 이미지 표시

- 사용 API: GET /recipe
- 관련 파일: services/recipe_service.py, utils/ingredient_translator.py

2. Redis 캐시 구조 요약

상황	캐시 키	내용	TT L
감정 분석 결과	emotion:{입력값}	GPT 음식 키워드 리스트	10 분
음식점 리스트	store:{음식명}	요기요 음식점 JSON 결과	10 분
사용자 세션	session:{user_id}	로그인 사용자 정보, JWT	1 일

3. 전체 기능별 시퀀스



📌 1. SNS 로그인

목적: 사용자 인증 및 JWT 발급

1. [프론트] 사용자가 SNS 로그인 버튼 클릭
2. [SNS 서버] 인증 코드 발급 후 Redirect URI 호출

3. [프론트] 인증 코드 수신 후 백엔드에 전달 (/auth/oauth/callback)
 4. [백엔드] SNS API를 통해 사용자 정보 조회
 5. [백엔드] 내부 사용자 DB에 등록 여부 확인
 6. [백엔드] JWT 토큰 생성 (액세스, 리프레시)
 7. [Redis] 사용자 세션 저장
 8. [프론트] JWT 토큰 수신 및 로컬 저장 (localStorage)
-

2. 감정 입력 → GPT 추천 키워드

목적: 감정 또는 상황에 따라 GPT가 어울리는 음식 키워드 추천

1. [프론트] 사용자 감정 입력 (예: 우울할 때 먹고 싶은 음식)
 2. [프론트] /recommendations/emotion API 호출
 3. [백엔드] 감정 텍스트를 GPT 프롬프트로 가공
 4. [GPT API] 감정에 어울리는 음식 키워드 응답
 5. [백엔드] 추천 결과 Redis 캐시 저장
 6. [프론트] 추천 음식 리스트 표시
-

3. 음식 키워드 선택 → 요기요 음식점 검색

목적: 사용자가 선택한 음식 키워드로 실제 음식점 검색

1. [프론트] 음식 키워드 클릭 (예: “피자”)
 2. [프론트] /restaurants/search?keyword=피자 호출
 3. [백엔드] 요기요 비공식 API 또는 크롤러로 음식점 정보 수집
 4. [백엔드] 평점, 리뷰 등 기준으로 정렬/필터링
 5. [Redis] 음식점 리스트 캐시 저장
 6. [프론트] 음식점 리스트 및 썸네일 표시
-

4. 음식 클릭 → 레시피 및 재료 정보 - 추후 완성 예정

목적: 사용자가 선택한 음식에 대한 레시피 제공

1. [프론트] 음식점 또는 음식 클릭 → 상세 요청
 2. [프론트] /recipes?food_name=김치찌개 호출
 3. [백엔드] Spoonacular API 또는 내부 DB에서 레시피 검색
 4. [백엔드] 영어 재료명 → 한글 번역
 5. [백엔드] 재료 + 조리법 구성 후 반환
 6. [프론트] 레시피 화면에 출력
-

5. 추천 결과 재사용 (캐시 기반)

목적: 동일 감정/키워드 요청 시 캐시 활용으로 속도 향상

1. [프론트] 이전에 입력한 감정 또는 키워드로 다시 요청

2. [프론트] /recommend_result/cached?key=emotion:우울해요 호출
3. [백엔드] Redis에서 캐시된 추천 결과 탐색
4. [프론트] 캐시된 결과 즉시 렌더링

📌 6. 사용자 위치 기반 검색 (선택 기능)

목적: 사용자의 현재 위치 근처 음식점 또는 카테고리 기반 탐색

1. [프론트] 사용자 위치 정보 수집 (latitude, longitude)
2. [프론트] /locations/nearby?lat=...&lng=... 호출
3. [백엔드] 위치 기반 검색 알고리즘 실행
4. [백엔드] 주변 음식점 또는 인기 메뉴 추천
5. [프론트] 결과 표시

4. ERD 구조 요약

✅ 주요 테이블 목록 및 설명

테이블명	설명	주요 컬럼	관계
users	사용자 정보 저장	id, email, full_name, is_active	1:N ↔ recommendations
auth_user	Django 기본 사용자 모델	username, email, is_staff	관리용
gomgom_ai_recommendation	감정 기반 추천 결과	input_text, recommended_store_chars, matched_restaurant_id	↔ users (user_id FK)
recommendations	음식 추천 기록	user_id, recipe_id, score	↔ users, recipes
recipes	레시피 상세 정보	name, instructions, difficulty, servings	1:N ↔ recommendations
ingredients	재료 기본 정보	name, price, unit, category	1:N ↔ ingredient_ko
ingredient_ko	재료 한글 번역 정보	eng_name, kor_name	↔ gomgom_ai_recipe

테이블명	설명	주요 컬럼	관계
gomgom_ai_ingredient	추천 결과 연동 재료	name, price, ko_info_id	↔ ingredient_ko
gomgom_ai_recipe	GPT 기반 추천용 레시피	title, description, category, keywords	외부 레시피 연계
locations	위치 기반 음식점 정보	name, latitude, longitude, category	위치 기반 추천 가능
auth_group, auth_permission	Django 내장 권한 시스템	-	관리 목적
django_session	세션 관리	session_key, session_data	Redis 또는 DB 기반
django_migrations	마이그레이션 이력	app, name, applied	Alembic 유사
django_content_type	Django 앱 모델 정보	app_label, model	admin UI 연계

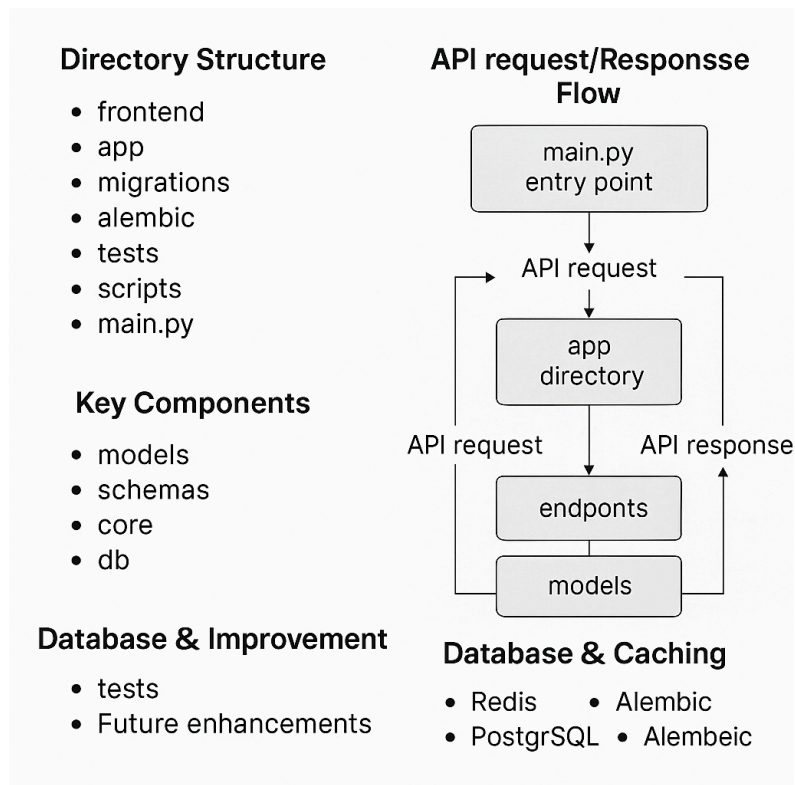
🗺️ 핵심 관계 구조 (요약)

```

[users]
├──< gomgom_ai_recommendation
│   ├──< gomgom_ai_recipe
│   │   ├──< gomgom_ai_ingredient
│   │   │   └──< ingredient_ko
[recipes]
├──< recommendations
│   └──< users

```

5. 기술 스택 기반 특이사항



-
- **FastAPI** → 사용자 정의 모델은 `gomgom_ai_` 접두어 사용
- **Redis** → 세션 저장소 및 캐시 레이어로 동시 활용됨
- **다국어 재료 처리** → `ingredient_ko`, `kor_name`, `eng_name` 구조 활용

6. 추후 완성예정 활용 예시 시나리오

- **감정에 따른 음식 레시피 추천**
Emotion → Recipe Recommendation: `gomgom_ai_recommendation.input_text` → `gomgom_ai_recipe` OR `recipes`
- **식재료 가격 비교**: `gomgom_ai_ingredient` → `price`, `unit`

작성자: 민예린

버전: 1.0

최종 수정일: 2025-06-18

깃허브 주소: <https://github.com/feed-mina>